

Peer Discovery and Membership in a Fully Distributed Peer-to-Peer Chat System

SJSU CMPE 275 Enterprise Applications — Final Project Report

1st Himanshu Jain 2nd Abhishek Jha 3rd Asim 4th Ali
San Jose State University San Jose State University San Jose State University San Jose State University
San Jose, USA San Jose, USA San Jose, USA San Jose, USA

Project Repository: https://github.com/thatoneguy01/PeerChat/tree/main/peer_discovery

Abstract

We will be showing you both the design as well as the ‘Implementation Project’ of our Peer Discovery Membership module in PeerChat, a Decentralized P2P Chat System. Due to the lack of centralized authority, achieving an accurate and consistent view of who the members of a given chat room are, across different peers, becomes very challenging. To solve this problem, we chose to implement an “Event Sourced State Machine” Architecture where the Gossip Protocol for Peers to Communicate with each other is based on “Websockets” and included the acquisition of failure detection via a “SWIM” style approach along with a Public Key Distribution Model based on “Trust On First Use”. Furthermore, we will discuss some of the challenges we faced while building and testing the project as related to various limitations of modern computing hardware including but not limited to the fact that many modern laptops do not have physical LAN ports and also how many wireless environments have limitations (e.g., AP Isolation) or configuration issues (e.g., Firewall) that could hinder our testing or use of the application.

I. INDIVIDUAL CONTRIBUTIONS

Member	Contributions
Himanshu	• Network layer: <code>DiscoveryNode</code>, <code>DiscoveryConfig</code>, <code>net_utils</code> • Wire protocol and envelope codecs (<code>protocol.py</code>) • Bootstrap handshake (<code>bootstrap.py</code>) • Gossip dispatcher (<code>gossip.py</code>) • Event-sourced membership state machine: <code>append-only</code> <code>MembershipEventLog</code>, <code>MembershipSnapshot</code> with transition guards, <code>DuplicateGuard</code> • <code>DurabilityManager</code> for crash recovery • End-to-end integration with Distribution’s <code>BroadcastNode</code> • Led the testing part
Abhishek	• SWIM-style presence detection: <code>PresenceManager</code>, <code>HeartbeatManager</code> • Trust-on-first-use security integration: <code>lazy_register_pubkey</code> hook • History backfill handshake lifecycle • <code>MembershipCoordinator</code> single-writer logic with join validation • <code>EventNotifier</code> subscriber fan-out, <code>MembershipService</code> facade, <code>JoinLifecycleTracer</code> • Verified API guarantees between Peer Discovery and security modules • Authored the final project report
Asim	• Cross-team integration contract test suites: <code>test_distribution_team_contract.py</code>, <code>test_history_team_contract.py</code>, <code>test_security_team_contract.py</code> • Verified API guarantees between Peer Discovery and Distribution, History modules
Ali	• Harden presence debug snapshots

II. INTRODUCTION

In traditional client server designs a central database acts as “the source of truth” when answering the question “who is in the room now?”. In contrast, in the decentralized peer-to-peer (P2P) chat system known as PeerChat, each peer must maintain their own answer to the same question. Furthermore, at all times and in all cases, regardless of node failure (unexpected node failure), network partitions, or asynchronous gossip cycles, all peers’ answers need to be consistent from an operational point of view.

Additionally, without a reliable member management function, the entire application stops functioning: the Message Distribution function fails to know what peers will receive messages; the History function fails to know which message backlog to replay when a peer joins after being absent; the UI would show an out-of-date roster; and, the Security function would have no method by which to reliably secure the proper public keys for encrypting payloads.

Our module provides strong guarantees:

- A node can join the local area with information of only a single seed node, then connect to other members in the group and create complete knowledge of their existence and public key information.
- Membership events (join, leave, suspect, timeout) are delivered exactly once and idempotently to all peers.
- Peering with an active fan-out will incidentally remove a peer from that list if the peer stops sending heartbeats after a period of time.
- Restarts are inexpensive because restarting nodes pick up where they left off in a local state, rather than needing to replay all events since genesis.
- Membership state for any given node is deterministically derived from the event log.

III. HIGH-LEVEL ARCHITECTURE

Peer Discovery serves as a mediator for the network transport layer from the UI/Security layers. In order to avoid the instability caused by having several networking stacks run at once (which happened in previous attempts), our networking layer works as a kind of adapter and uses the `BroadcastNode` provided by the Message Distribution team.

The architecture is divided into two planes:

- 1) **Control Plane:** The process is currently active, and refers to a single writer protocol. The control plane consists of an Event Log containing a series of events that only gets appended to; a Coordinator that is single and unique; a Snapshot view; and a Notifier.
- 2) **Wire Plane:** The responsibilities of this plane include gossip, heartbeat, and bootstrap. Every event is serialized into a JSON object inside `Message.content` and passed on to the distribution layer.

IV. MEMBERSHIP STATE MACHINE

A. Event-Sourced Source of Truth

In order to ensure determinacy across all nodes, the design uses an event sourcing approach. `MembershipEventLog` is the name for the log which is both append-only and single writer in nature. All events, whether related to peers joining or timing-out, are appended as `MembershipEvent` records having a monotonically increasing number for each term. `MembershipEventLog` is the only persistent data structure. Room state or the `MembershipSnapshot` is derived as a pure function of this log.

B. State Transitions

Each member passes through a tightly controlled state machine: `JOIN_ACCEPTED` → `JOINING` → `BACKFILLING` → `ACTIVE` ↔ `SUSPECTED` → `DISCONNECTED`.

State transitions are rigorously controlled. For instance, `ACTIVE` is the only state in which Message Distribution routes live chat traffic for that member. `JOINING` and `BACKFILLING` are "held" states where the member has been discovered but is still getting message history replayed.

C. Idempotency and Duplicate Suppression

As a result of being redundant, the same `JOIN_ACCEPTED` event could be received from several peers. Re-applying it would destroy the snapshot. The solution to this problem involves the use of the `DuplicateGuard` module, which uses the following key: (`originator`, `seq_no`, `event_type`, `user_id`) when performing duplicate detection. This particular key structure is also utilized both in the LRU caching layer for the network layer as well as the persistent guard for the log layer.

V. NETWORK LAYER AND PROTOCOL

A. Consolidated Transport and Wire Protocol

The network layer makes use of the `BroadcastNode` running at port number 5678, which belongs to the Distribution group. Each membership message is encapsulated within the message itself in JSON format and includes fields such as `type`, `sender_public_key_pem_b64`, and `payload`. Four different types of envelopes are used:

- 1) `discovery_join_request` — sent from a joining node to a seed node. Payload carries only the joiner's display name.
- 2) `discovery_join_response` — sent from the seed back to the joiner. Payload carries the full event-log replay and an acceptance decision.
- 3) `discovery_gossip` — broadcast from any node to all peers. Payload carries a single serialized `MembershipEvent`.
- 4) `discovery_heartbeat` — broadcast from any node to all peers. Payload is empty; the sender's address in `Message.sender` carries all necessary identity information.

Message discrimination is handled via a lightweight substring check: the function `is_discovery_message()` looks for the literal `"type": "discovery_"` in the raw content string. This is fast and collision-free because chat messages are opaque user text. Structural validation occurs downstream in `decode_discovery_envelope`.

B. Bootstrap Protocol

With a new peer (joiner), only the IP address and the port number of the seed are known at first because nothing about the seed's public key is known. Bootstrap starts by sending a `JOIN_REQUEST` signed in plaintext. Seed's node uses a `pre_verify_hook` in order to obtain the public key of the joiner from the message body. Once accepted, seed sends a `JOIN_RESPONSE`, which includes an event log replay for the entire room in an encrypted and signed form.

The full bootstrap handshake proceeds in the following steps:

- 1) The joiner registers a `threading.Event` keyed by the seed's `host:port`. This is the synchronization primitive that the bootstrap thread will block on.
- 2) The joiner constructs a `JOIN_REQUEST` envelope containing its display name and its public key PEM, wraps it in a Distribution Message, and calls `BroadcastNode.send_to_peer()` for point-to-point delivery.
- 3) The joiner blocks on the Event with a configurable timeout (5.0 seconds by default). The entire bootstrap is synchronous from the perspective of the Flask HTTP request.
- 4) On the seed side, the `pre_verify_hook` fires before signature verification. It extracts the joiner's public key from the envelope and plants it in the peer registry (trust-on-first-use).
- 5) Distribution's `verify()` now succeeds because the key was just planted. The message is accepted and ACKed.
- 6) The seed's coordinator calls `service.join_member()`, which appends a `JOIN_ACCEPTED` event and projects it into the snapshot.
- 7) The seed constructs a `JOIN_RESPONSE` containing the full event-log replay and the seed's own public key. Because the joiner's key is now in the registry, this response is **encrypted-then-signed**.
- 8) On the joiner side, the same hook plants the seed's key. Distribution decrypts and verifies.
- 9) The joiner applies the snapshot via `service.apply_remote_snapshot()`, hydrating its local event log and membership state.
- 10) The pending `threading.Event` is set, and `attempt_bootstrap()` returns `True`. The joiner is now a full participant.

If the seed is unreachable (firewall, wrong NIC, AP isolation), the Event never fires and the joiner logs `bootstrap_no_response` and runs isolated.

C. Gossip Dissemination

The new delta for membership issued by the local coordinator is packaged using a `discovery_gossip` wrapper before being multicast to every node. Gossip protocol contains cycle-breaking mechanisms where the event having "remote" as its source is not gossiped again. Local originator does its own gossiping and once done, recipients apply this event. This limits the number of hops to be just one per each node.

VI. FAILURE DETECTION (SWIM-STYLE)

Since peers may disconnect ungracefully, we implemented a failure detector modeled on the SWIM protocol. A `HeartbeatManager` broadcasts a heartbeat envelope every 5 seconds. In tandem, a `presence-tick` thread evaluates peer liveness every 1 second.

The failure detector operates in two phases:

- 1) **Suspected Phase:** If a peer misses heartbeats beyond a 15-second threshold, it is moved from `ACTIVE` to `SUSPECTED`.
- 2) **Timeout Phase:** If the `SUSPECTED` peer does not issue a heartbeat for an additional 15 seconds, the manager fires a `DISCONNECT_TIMEOUT` event, completely dropping the peer from the active routing set.

If a heartbeat is received while a peer is `SUSPECTED`, they immediately bounce back to `ACTIVE`, allowing the network to gracefully tolerate temporary network jitters.

VII. SECURITY AND TRUST-ON-FIRST-USE

The system mandates the use of end-to-end encryption and authentication through the verification of signatures with asymmetric keys. With the absence of a Key Management Service (KMS), our solution is to implement the Trust On First Use (TOFU) model.

Each discovery envelope will carry the public key of the sender in PEM format. Incoming messages are intercepted through the `DiscoveryNode.lazy_register_pubkey` hook in Distribution's `pre_verify_hook` interface. In effect, the initial message from a sender will register the public key before any verification happens. Thus, we have distributed public keys together with the admission events.

VIII. DURABILITY AND CRASH RECOVERY

The `DurabilityManager` writes periodic checkpoints so a restarting node does not need to replay the entire event log from genesis. A checkpoint is a single JSON file containing the serialized snapshot, the log tail (events after the snapshot's last applied sequence number), the current term, and the next sequence number.

Writes are atomic: the manager writes to a temporary file, calls `fsync()`, and then uses `os.replace()` to atomically swap the file into place. The last two checkpoints per room are retained so recovery can fall back to the previous one if the newest is corrupt.

The recovery path proceeds as follows:

- 1) Load the latest checkpoint and deserialize the `MembershipSnapshot`.
- 2) Restore the `DuplicateGuard` from the same checkpoint, ensuring that replayed gossip events cannot double-apply.
- 3) Defensively replay any events in the log tail that fall after the snapshot's last applied sequence number.
- 4) Resume normal operation.

The key insight underlying this design is that only the event log needs to be durable. The snapshot is a pure function of the log and can always be reconstructed by replay. This is the same property exploited by systems such as Apache Kafka, where events are the source of truth and materialized views are treated as caches [3].

IX. HISTORY BACKFILL HANDSHAKE

A newly joined peer cannot display chat messages that were exchanged while it was offline. The membership state machine manages this through a three-event lifecycle:

- 1) When `JOIN_ACCEPTED` is committed, the coordinator fires `HISTORY_BACKFILL_STARTED`, and the member transitions from `JOINING` to `BACKFILLING`. While in this state, the member exists in the room but is **not** included in Distribution's fan-out set — live chat traffic is held.
- 2) The History team's handler (registered via `service.register_history_handler()`) replays missed messages to the joiner over Distribution's `send_to_peer()`.
- 3) When the replay is complete, the History team calls `service.complete_history_backfill(user_id)`, which logs `HISTORY_BACKFILL_COMPLETE`. The member transitions to `ACTIVE` and becomes a full fan-out target.

When no History handler is registered (the demo path), our `_auto_complete_backfill_handler` immediately fires both the start and complete events, promoting the joiner to `ACTIVE` without delay. This fallback allows two-laptop integration tests to run without the History module being wired in.

X. SUBSCRIBER API AND EVENT NOTIFICATION

The `EventNotifier` provides a subscribe-with-from-version API that eliminates the classic snapshot-subscription race condition. Callers subscribe by specifying a `from_version` parameter:

- 1) The notifier synchronously replays every event with `version > from_version`, ensuring no events are missed between a snapshot read and the subscription.
- 2) After the replay, subsequent events are delivered live as they are committed by the coordinator.

Distribution's `MembershipRouter` uses this mechanism: it takes a snapshot at version `v`, subscribes with `from_version=v`, and is guaranteed to miss nothing in between. The dispatch is single-threaded and synchronous, which is a documented caveat — a slow callback will delay all subscribers behind it — but is acceptable at control-plane traffic volumes.

XI. INTEGRATION WITH OTHER MODULES

The Peer Discovery module serves as the foundational integration point for the rest of the application:

- **Distribution:** We have provided the `MembershipRouter`, which takes care of fan-out messages to `ACTIVE` members only, and we have provided `pre_verify_hook` to register keys in an exception-safe manner.
- **Security:** We have taken care of the distribution of the public cryptographic key to ensure that the Security module can safely encrypt the payload for the respective receiver.
- **History:** We orchestrate the `BACKFILLING` state, invoking the History team's replay callable before officially promoting a joiner to the `ACTIVE` state.

XII. TESTING AND VALIDATION

A. Automated Test Suite

We maintained a suite of 92 unit and integration tests across three test directories:

- **Membership tests** (`peer_discovery/membership/tests/`) — event log append ordering, snapshot projection correctness, transition guard enforcement, duplicate guard idempotency, durability checkpoint roundtrip.
- **Integration tests** (`peer_discovery/membership_integration/tests/`) — coordinator single-writer discipline, notifier replay-then-live semantics, service facade end-to-end flows.
- **Network tests** (`peer_discovery/network/tests/`) — protocol codec roundtrip, gossip dispatcher LRU dedup, heartbeat scheduling, and a two-node end-to-end test through a stubbed `BroadcastNode`.

All 92 tests pass consistently when executed via `python3 -m pytest peer_discovery/ -q`.

B. Manual Two-Laptop Validation

Manual validation was performed on 2026-05-18 across two physical machines on a `10.0.0.0/24` LAN:

- The seed booted cleanly with no `bootstrap_peers`, becoming a one-member room.
- The joiner connected with the seed’s IP address only (no pre-shared public key). The `pre_verify_hook` successfully planted the joiner’s public key on the seed side.
- The `JOIN_RESPONSE` returned encrypted-then-signed with a full 6-event log replay. The joiner entered `ACTIVE` within approximately 200 milliseconds.
- Bidirectional heartbeats were sustained for over 30 minutes without a single drop.
- Live chat messages flowed in both directions, signed and verified end-to-end.

C. End-to-End Backend Test

A dedicated `test_backend_e2e.py` exercises the full Distribution + Discovery integration with two stubbed `BroadcastNode` instances communicating in-process, verifying the `JOIN` handshake, gossip propagation, and the trust-on-first-use bootstrap path without requiring real `WebSocket` connections.

XIII. PRACTICAL CHALLENGES AND LIMITATIONS

Building a peer-to-peer system on consumer hardware and enterprise networks introduced severe practical roadblocks that shaped our final design.

A. Transport Collision and Port Exhaustion

In the beginning, the Peer Discovery and Distribution modules each had their own transport server operating in parallel on a separate TCP port. This design choice led to port conflicts, half-open connection issues, and “Incoming frame size” parsing failures when the chat messages happened to get directed at the wrong port for discovery purposes. The key realization was to bring both stacks together under a `BroadcastNode` component.

B. Modern Hardware: No LAN Ports

In our process of manual two-laptops integration testing, we encountered a problem that exists because of a hardware limitation – the current generation of laptops does not have an Ethernet LAN port to allow a connection between the two computers.

C. Wi-Fi AP Isolation and Firewalls

Having to use Wi-Fi, we tried using Wi-Fi networks provided by the university or the public. Enterprise Wi-Fi networks, however, use Access Point (AP) Isolation (also known as Client Isolation). With AP Isolation, no communication is allowed between two devices attached to the same wireless network. While both laptops would have obtained IP addresses belonging to the same subnet (for example, `10.0.0.x/24`), pinging and `WebSocket` connection attempts would have been prevented.

Additionally, Windows Defender and macOS firewalls blocked unsolicited incoming `WebSocket` connections on port 5678, manifesting as `bootstrap_no_response` timeouts.

Mitigation: In order to avoid using AP Isolation, we had to set up our own private hotspot and configure specific inbound firewall rules within our host OSs. Moreover, we came across problems in which multi-NIC hosts (hosts with open VPN tunnels, for instance) led to the incorrect advertisement of `get_lan_ip()`. This underlined the need for the use of “bind vs advertise”, whereby the application would listen at `0.0.0.0`, yet advertise another IP address explicitly to the cluster.

XIV. CONCLUSION

The Peer Discovery and Membership module has effectively managed to develop a powerful decentralized membership state machine to drive the functionality of the PeerChat application. The use of event sourcing, deterministic gossip, SWIM heartbeats, and TOFU cryptographic bootstrap allows the system to maintain its internal member status independently from any central servers. The most important design choice was made when we decided to integrate our transport layer into the Distribution group, which led us to better stability and allowed solving many issues related to networking problems caused by isolated Wi-Fi networks.

REFERENCES

- [1] A. Das, I. Gupta, and A. Motivala, “SWIM: Scalable Weakly-consistent Infection-style Process Group Membership Protocol,” in *Proc. Int. Conf. Dependable Syst. Networks (DSN)*, 2002.
- [2] L. Lamport, “Time, Clocks, and the Ordering of Events in a Distributed System,” *Commun. ACM*, vol. 21, no. 7, pp. 558–565, 1978.
- [3] J. Kreps, “The Log: What every software engineer should know about real-time data’s unifying abstraction,” *LinkedIn Engineering*, 2013.